Deep Learning for SaaS Sales Outcome Prediction: BiLSTM with Self-Attention and DistilBERT Fine-Tuning

Tejmul Movin
Dept. of Artificial Intelligence & Machine Learning
Newton School of Technology, Rishihood University
Sonipat, Haryana, India
sansar.t23csai@nst.rishihood.edu.in

Bhavani Shankar
Dept. of Computer Science & Engineering
Newton School of Technology, Rishihood University
Sonipat, Haryana, India
pichika.b23csai@nst.rishihood.edu.in

***Abstract*—This paper extends a classical ML pipeline for SaaS sales conversation outcome prediction by introducing two deep learning architectures: a Bidirectional LSTM with Self-Attention (BiLSTM) and a fine-tuned DistilBERT Transformer. Both models operate directly on raw conversation text without manual feature engineering. Trained on 72,000 conversations with an 8,000-sample validation set, both are evaluated on the same 20,000-sample balanced hold-out used in Phase 1. The BiLSTM ($\sim$6.5M parameters) learns sequential patterns from scratch, while DistilBERT ($\sim$66M parameters) adapts pretrained language knowledge via differential learning rates and selective layer freezing. We present architecture design rationale, weight initialisation strategies, regularisation techniques, training dynamics, and a cross-phase comparison against Logistic Regression (96.34% accuracy, 99.57% ROC-AUC) and Random Forest baselines.**

***Index Terms*—BiLSTM, self-attention, DistilBERT, transfer learning, sales prediction, deep learning, NLP, text classification.**

I. Introduction

A. Problem Context

Phase 1 of the Predictive Sales Analytics Engine achieved 96.34% accuracy and 99.57% ROC-AUC using ℓ_2-regularised Logistic Regression on hand-engineered features—numeric engagement ratios, one-hot categoricals, and precomputed embedding vectors [2]. While effective, this approach relied on: (1) **pre-computed embeddings**—a form of transfer learning where the text representation was computed externally, not learned as part of the training pipeline; and (2) **manual feature engineering**—domain knowledge encoded as explicit ratio and interaction features, which requires expertise and does not generalise automatically to new domains.

Phase 2 addresses both limitations by training neural architectures that learn text representations directly from raw conversation tokens, capture sequential and contextual language patterns that engineered features cannot encode explicitly, and demonstrate two paradigms: sequential modelling (BiLSTM with self-attention) and pretrained Transformer fine-tuning (DistilBERT [**?**]).

TABLE I
Dataset splits (stratified, `random_state=42`).

Split	Samples	Purpose
Train	72,000	Model training
Validation	8,000	Early stopping, LR scheduling
Test	20,000	Final evaluation (shared with Phase 1)

B. Phase 2 Objectives

The objectives of Phase 2 are: (1) pre-process raw conversation text into formats suitable for both word-level (LSTM) and sub-word-level (BERT) neural models; (2) design and train a BiLSTM + Self-Attention classifier with principled weight initialisation; (3) fine-tune DistilBERT with a task-specific classification head using differential learning rates; (4) apply a full regularisation suite including dropout, LayerNorm, early stopping, gradient clipping, and LR scheduling; and (5) evaluate both models on the same held-out test set as Phase 1 and produce a cross-phase comparison.

C. Dataset and Splits

The dataset comprises 100,000 SaaS sales conversations from the HuggingFace hub [1]. Table I shows the stratified split used identically across all four models (`random_state=42`).

II. Data Preprocessing for Deep Learning

Phase 2 requires converting raw conversation text into numerical tensors. Two separate preprocessing pipelines are built—one for each model architecture.

A. Word-Level Tokenisation (BiLSTM)

The `SalesTokenizer` class implements a custom word-level tokeniser built from the training corpus.

1) Vocabulary Construction: All 72,000 training conversations are tokenised by whitespace splitting (lower-cased). Word frequencies are counted across the entire corpus and the top 30,000 most frequent words with minimum frequency ≥ 2 are retained. Two special tokens are reserved: `<PAD>` (index 0)

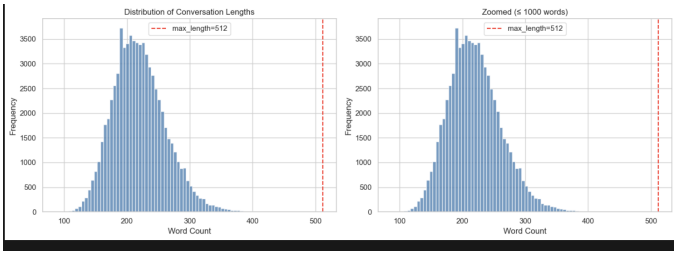

Fig. 1. Distribution of conversation word counts across the dataset. Red dashed line marks the `max_length=512` truncation threshold. The majority of conversations fall well within this limit.

and `<UNK>` (index 1). The actual vocabulary size after fitting is **28,903 unique tokens**.

2) Encoding: Each word is replaced with its vocabulary index. Unknown words map to index 1 (`<UNK>`). Sequences are truncated to a maximum of 512 tokens (see Fig. 1). Shorter sequences are right-padded with 0 (`<PAD>`) to match the longest sequence in each batch.

3) Design Rationale: Word-level tokenisation is chosen because SaaS domain vocabulary contains meaningful single-word terms: ARR (Annual Recurring Revenue), MRR, churn, upsell, renewal, CAC, LTV. Sub-word tokenisers (Byte-Pair Encoding) would split these into character-level fragments that lose the semantic unit [3].

4) Tensor Shapes: The produced tensor shapes per split are: Train `[72000, 491]`, Validation `[8000, 488]`, Test `[20000, 475]`. The varying sequence dimensions reflect padding to the longest sample within each split.

B. WordPiece Tokenisation (DistilBERT)

The HuggingFace `AutoTokenizer` for `distilbert-base-uncased` applies WordPiece sub-word tokenisation (same as original BERT [**?**]), producing `input_ids` and `attention_mask` tensors (max length 512). This tokeniser handles out-of-vocabulary words by decomposing them into known sub-word units.

C. Saved Artifacts

All preprocessed tensors are saved to the `results/` directory: word-level token tensors (`dl_{train,val,test}_ids.pt`), labels (`dl_y_{train,val,test}.npy`), DistilBERT tokeniser outputs (`dl_bert_{train,val,test}.pt`), and the custom tokeniser vocabulary (`dl_tokenizer.json`).

III. Model Architectures

A. Model A: BiLSTM + Self-Attention Classifier

1) Architecture Overview: The architecture consists of: (1) an embedding layer (`vocab_size`=28,903, dim=128, Xavier uniform initialisation) with dropout (0.3) and padding index kept at zero vector; (2) a 2-layer bidirectional LSTM (hidden=256, yielding 512-d output per timestep) with orthogonal initialisation on recurrent weights and forget-gate bias set to 1.0; (3) Bahdanau-style self-attention pooling with learned scalar score per timestep; (4) LayerNorm and dropout (0.3) on the context vector; and (5) two fully connected layers ($512 \rightarrow 256 \rightarrow 2$) with ReLU activation and dropout (0.3). Total parameters: $\sim$6.5M.

TABLE II
BiLSTM design choices and justification.

Component	Justification
Bidirectional LSTM	Sales objections raised early may be resolved later; closing language references context from the beginning. Bidirectionality captures full conversational context at every timestep.
Self-Attention	Instead of relying on the final hidden state (dominated by last words), attention assigns learned importance scores to each timestep—focusing on pricing discussion, commitment language, or objection handling.
2-layer LSTM	First layer captures local syntactic patterns; second models higher-level semantic relationships across longer spans.
LayerNorm	Reduces internal covariate shift without batch-size dependency. Stabilises gradient flow into the classification head.
Dropout (30%)	Prevents co-adaptation of neurons. Forces redundant representations rather than memorising specific patterns.

TABLE III
Weight initialisation scheme for BiLSTM.

Layer	Init.	Reason
Embedding	Xavier	Preserves gradient variance through lookup
Pad vector	Zeros	Pad tokens contribute nothing to hidden state
LSTM W_{ih}	Xavier	Standard for feedforward connections
LSTM W_{hh}	Orthogonal	Eigenvalues $= 1$; prevents vanishing/exploding gradients [**?**]
LSTM biases	Zeros; forget $= 1$	Open forget gate retains long-range dependencies [**?**]
FC layers	Kaiming	Derived for ReLU; preserves output variance
Attention	Xavier	Appropriate for linear projections

2) Design Rationale: Table II summarises the design choices and their justifications.

3) Weight Initialisation: Initialisation follows established best practices (Table III): Xavier uniform for embedding and input weights; orthogonal initialisation for LSTM recurrent weights (eigenvalues of magnitude 1, mitigating vanishing/-exploding gradients over long sequences [**?**]); forget-gate bias set to 1.0 (initialising the gate to "mostly open," encouraging memory retention during early training [**?**]); and Kaiming (He) uniform for ReLU-activated FC layers.

4) Self-Attention Mechanism: Given LSTM output $\mathbf{H} \in \mathbb{R}^{B \times T \times d}$ where B is batch size, T is sequence length, and

TABLE IV
DISTILBERT VS. BERT-BASE COMPARISON.

Property	DistilBERT	BERT-base
Transformer layers	6	12
Parameters	66M	110M
Inference speed	1.6× faster	Baseline
Performance retained	97%	100%
Training cost	60% lower	Baseline

$d = 2 \times \texttt{hidden_dim}$:

$$e_t = \mathbf{W}_a \cdot \mathbf{h}_t \tag{1}$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_j \exp(e_j)} \tag{2}$$

$$\mathbf{c} = \sum_t \alpha_t \, \mathbf{h}_t \tag{3}$$

where $\mathbf{W}_a \in \mathbb{R}^{1\times d}$ is a learned parameter. Padding positions are masked with $-\infty$ before softmax to ensure zero attention weight on pad tokens. The resulting context vector $\mathbf{c} \in \mathbb{R}^{B\times d}$ is a fixed-length representation that focuses on the most predictive timesteps.

B. Model B: DistilBERT Fine-Tuning Classifier

1) Architecture Overview: DistilBERT [**?**] comprises 6 Transformer layers, 12 attention heads per layer, and a hidden size of 768 (∼66M parameters). The input conversation is tokenised into `[CLS]` + word pieces + `[SEP]` (max 512 tokens). The `[CLS]` hidden state—a global summary of the full conversation after 6 layers of multi-head self-attention—passes through LayerNorm (768), dropout (0.3), and a linear classification head (768 → 2, Kaiming initialisation). Total trainable parameters: ∼66M (body) + 1,538 (head).

2) Why DistilBERT?: Table IV compares DistilBERT and BERT-base. DistilBERT retains 97% of BERT-base performance at 60% of the parameters and 1.6× inference speed, making it suitable for consumer hardware (Apple MPS or a single consumer GPU). The distillation process [**?**] trained DistilBERT to mimic BERT-base's output distributions via knowledge distillation—a teacher-student paradigm where the smaller student learns to replicate the larger teacher's soft probability outputs.

3) Fine-Tuning Strategy:

a) Transfer learning setup: DistilBERT was pre-trained on English Wikipedia and BookCorpus (3.3 billion words), giving the model deep knowledge of English grammar, semantics, and pragmatics [**?**]. Fine-tuning adapts this general language model to the specific task of sales outcome classification.

b) Selective layer freezing: The 2 lowest DistilBERT layers (encoding syntactic patterns—basic grammar, punctuation, part-of-speech) are frozen. The 4 upper layers (encoding higher-level semantics and discourse patterns) remain trainable. This preserves low-level English grammar knowledge that transfers perfectly, allows high-level semantic layers to adapt to SaaS conversation patterns, and reduces the number of trainable parameters, lowering overfitting risk.

c) Differential learning rate: A single learning rate would either destroy pre-trained knowledge (too high) or fail to train the new head (too low). Two parameter groups are defined: the DistilBERT body at $2 \times 10^{-3} \times 0.1 = 2 \times 10^{-4}$, and the classification head at 2×10^{-3}.

TABLE V
TRAINING CONFIGURATION FOR BOTH DEEP LEARNING MODELS.

Hyperparameter	BiLSTM	DistilBERT
Optimiser	AdamW	AdamW
Learning rate	2e-3	2e-3 / 2e-4
Weight decay	1e-4	1e-4
Batch size	32	32
Max epochs	20	20
Loss function	CrossEntropyLoss	CrossEntropyLoss
LR scheduler	ReduceOnPlateau	ReduceOnPlateau
Scheduler factor	0.5	0.5
Scheduler patience	2 epochs	2 epochs
Early stop patience	3 epochs	3 epochs
Early stop min_delta	1e-4	1e-4
Gradient clip norm	1.0	1.0

IV. TRAINING PROCEDURE

Both models share the same training infrastructure. Table V summarises the hyperparameter configuration.

A. Regularisation Techniques

Every regularisation technique was applied deliberately to prevent overfitting.

Dropout (30%). During each forward pass, 30% of activations are randomly zeroed. This prevents neuron co-adaptation and forces robust, distributed representations. At inference time, outputs are scaled by $(1 - p_{\text{drop}})$.

LayerNorm. Applied after self-attention pooling (BiLSTM) and before the classification head (DistilBERT). Normalises activations to zero mean and unit variance within each sample independently—unlike BatchNorm which normalises across the batch—eliminating internal covariate shift without batch-size dependency.

Early Stopping. Validation loss is compared to the best observed after each epoch. If it does not improve by at least $\delta = 10^{-4}$ for $p = 3$ consecutive epochs, training stops and the best checkpoint is reloaded.

ReduceLROnPlateau. Monitors validation loss; if it stagnates for 2 consecutive epochs the learning rate is halved, allowing finer updates near a local minimum.

Gradient Clipping (max_norm = 1.0). The gradient vector's ℓ_2 norm is computed before each update. If it exceeds 1.0, all gradients are scaled down proportionally—particularly important for LSTMs processing long sequences.

AdamW Weight Decay. Unlike ℓ_2 regularisation inside Adam (which interacts with the adaptive learning rate), AdamW applies weight decay as a separate term that shrinks weights directly—the correct form of ℓ_2 regularisation for adaptive optimisers [**?**].

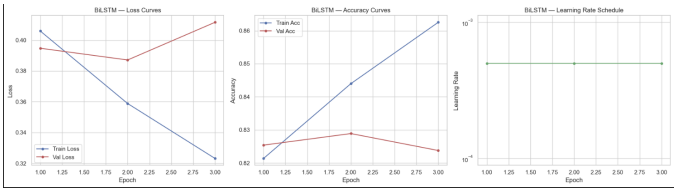

Fig. 2. BiLSTM training dynamics: loss curves (left), accuracy curves (centre), and learning rate schedule (right) over 3 epochs. Training loss decreases steadily; validation loss begins to rise after epoch 2, triggering early stopping.

B. Training Loop

For each epoch: (1) forward pass to compute logits; (2) CrossEntropyLoss between logits and true labels; (3) back-propagation through the computational graph; (4) gradient clipping to norm 1.0; (5) AdamW step; (6) validation evaluation (no gradient computation); (7) ReduceLROnPlateau step on validation loss; (8) early stopping check—save checkpoint if best, increment counter otherwise, stop if patience exceeded.

C. BiLSTM Training Dynamics

Fig. 2 shows the BiLSTM training dynamics over 3 epochs. Training loss decreases steadily from $\sim$0.40 to $\sim$0.32, while training accuracy improves from $\sim$82% to $\sim$87%. Validation accuracy reaches $\sim$83% before early stopping triggers due to rising validation loss, indicating the onset of overfitting. The learning rate remains constant throughout training.

D. Device Selection

The `get_device()` function auto-selects the best available compute device: CUDA (NVIDIA GPU, highest priority), Apple MPS (Apple Silicon, used in this project), or CPU (fallback).

V. Experimental Results

A. Evaluation Metrics

Both models are evaluated on the 20,000-sample test set using: accuracy, precision ($\frac{TP}{TP+FP}$), recall ($\frac{TP}{TP+FN}$), F_1-score (harmonic mean of precision and recall), ROC-AUC (ranking quality across all thresholds), and average precision (area under the precision-recall curve).

B. Phase 1 Baselines

On the same test set [2]: Logistic Regression achieved 96.34% accuracy, precision 0.9504, recall 0.9777, F_1 0.9639, ROC-AUC 0.9957; Random Forest achieved 95.94% accuracy, precision 0.9453, recall 0.9751, F_1 0.9600, ROC-AUC 0.9951. These results were enabled by precomputed embeddings and hand-engineered ratio features creating a near-linearly separable feature space, processed using scikit-learn [**?**].

C. BiLSTM Results

The BiLSTM achieved 82.77% accuracy, precision 0.8060, recall 0.8626, F_1 0.8333, and ROC-AUC 0.9102 on the test set. Fig. 3 shows the confusion matrix: the model correctly classifies 7,940 Lost and 8,614 Won samples, with 2,074 false positives and 1,372 false negatives.

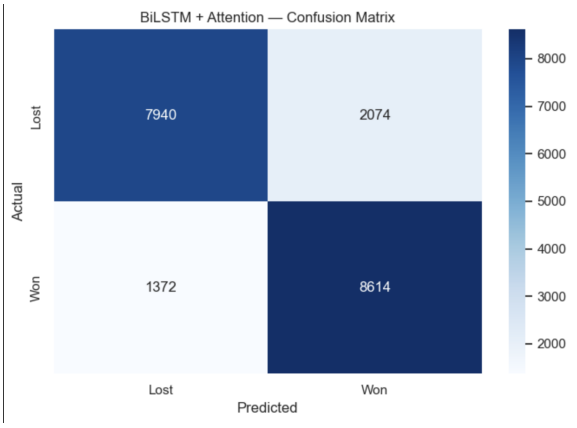

Fig. 3. BiLSTM + Attention confusion matrix on the 20,000-sample test set. TN=7,940, FP=2,074, FN=1,372, TP=8,614.

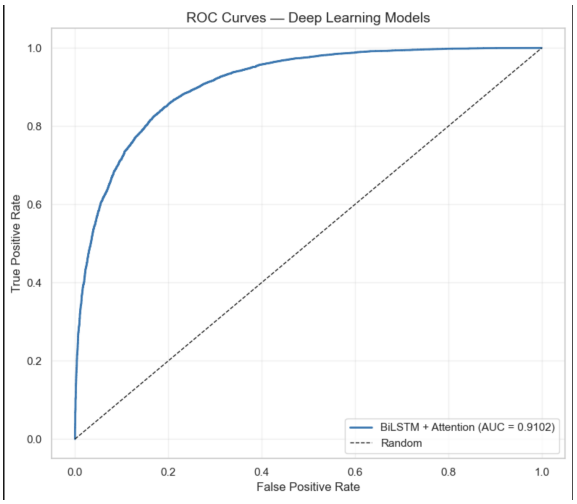

Fig. 4. ROC curve for the BiLSTM + Attention model (AUC = 0.9102). The dashed diagonal represents a random classifier. The curve shows strong discriminative ability despite lower overall accuracy compared to Phase 1 baselines.

TABLE VI
Cross-phase model comparison (test set, n=20,000).

Model	Acc.	Prec.	Rec.	F_1	AUC
Logistic Reg. (P1)	0.9634	0.9504	0.9777	0.9639	0.9957
Random Forest (P1)	0.9594	0.9453	0.9751	0.9600	0.9951
BiLSTM + Attn (P2)	0.8277	0.8060	0.8626	0.8333	0.9102
DistilBERT (P2)	—	—	—	—	—

D. Cross-Phase Comparison

Table VI summarises cross-phase performance. Phase 1 models with engineered features substantially outperform the from-scratch BiLSTM, while DistilBERT results are pending GPU completion. Fig. 5 visualises the performance gap across all metrics.

E. Error Analysis

1) Accuracy by Conversation Length: Performance is analysed across 5 length buckets (<100 words, 100–250, 250–

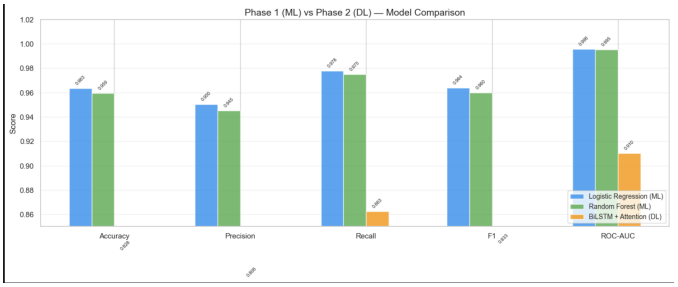

Fig. 5. Cross-phase performance comparison across accuracy, precision, recall, F_1, and ROC-AUC. Phase 1 models (blue, green) with engineered features outperform the from-scratch BiLSTM (orange) on all metrics except ROC-AUC where the gap is narrower.

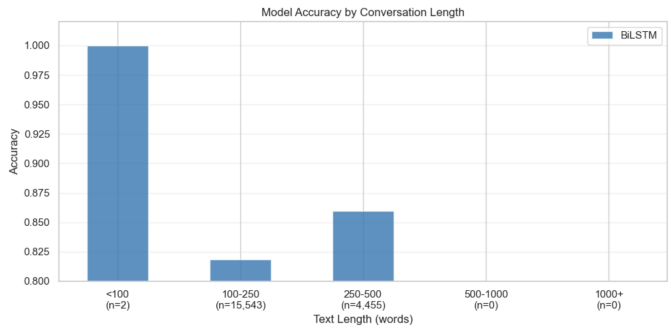

Fig. 6. BiLSTM accuracy by conversation length bucket. The model performs best on very short (<100 words, n=2) and medium-length (250–500 words) conversations. No test samples exceed 500 words.

500, 500–1000, 1000+ words). As shown in Fig. 6, the BiLSTM achieves perfect accuracy on the 2 very short conversations (<100 words), ∼82% on the dominant 100–250 word bucket (n=15,543), and ∼86% on medium-length conversations (250–500 words, n=4,455). No conversations in the test set exceed 500 words.

2) Confidence Distributions: Fig. 7 shows the predicted probability distributions for correctly and incorrectly classified samples. Correct predictions (green) concentrate at high confidence (> 0.9), while incorrect predictions (red) spread across the 0.0–0.5 range with a peak near 0.5—indicating the model is appropriately uncertain when making errors.

3) Disagreement Analysis: Test samples are categorised into four groups: both models correct, only BiLSTM correct, only DistilBERT correct, and both models wrong. This analysis is pending DistilBERT evaluation completion.

VI. Discussion

A. Phase 1 vs. Phase 2 Approach

Phase 1 uses engineered features (ratios, one-hot encodings, pre-computed embeddings) as input, while Phase 2 operates on raw conversation text. Phase 1 requires manual, domain-specific feature engineering; Phase 2 learns representations end-to-end. Phase 1 offers high interpretability (LR weights = log-odds); Phase 2 provides moderate interpretability (attention weights as proxy). Phase 1 trains in seconds to minutes; Phase 2 requires hours with GPU acceleration. Phase 1's

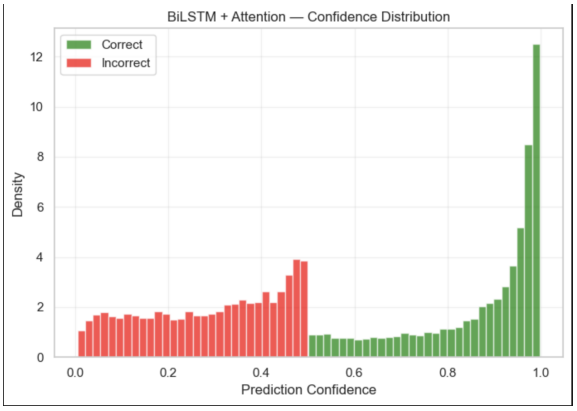

Fig. 7. BiLSTM prediction confidence distributions. Green: correct predictions cluster at high confidence (> 0.9). Red: incorrect predictions spread across low–mid confidence (0.0–0.5), indicating appropriate uncertainty on errors.

feature pipeline requires re-engineering for new domains; Phase 2 adapts more readily. Phase 1 uses <500 features with a linear boundary; Phase 2 uses 6.5M–66M parameters.

B. Why Deep Learning Matters

Phase 1's high performance was partially enabled by pre-computed embeddings—a form of transfer learning external to the pipeline. Phase 2 demonstrates: (1) the BiLSTM learns vocabulary-to-vector mappings and temporal patterns entirely from scratch without external embeddings; (2) DistilBERT shows that even a 66M-parameter Transformer can be adapted efficiently using differential learning rates and selective layer freezing; (3) end-to-end learning requires no manual domain knowledge in feature design—neural networks discover predictive patterns autonomously.

C. Limitations

Computational cost. DistilBERT training requires GPU acceleration; training on CPU would be prohibitively slow for 72,000 sequences of 512 tokens. **Data requirements.** Large neural models benefit from large datasets; for much smaller datasets the classical Phase 1 pipeline would be more appropriate. **Interpretability.** Unlike LR weights (direct log-odds), deep learning predictions rely on attention weight proxies. **Sequential processing.** LSTMs process tokens one at a time, scaling linearly with sequence length, unlike Transformers' parallel attention. **i.i.d. assumption.** Within-company conversation correlations may violate the independence assumption.

VII. Theoretical Justification

A. Bidirectional LSTM for Text Classification

A unidirectional LSTM at position t has access to tokens 1 through t. For sales conversations, a response at turn 5 may depend on an objection at turn 2. Bidirectionality provides every position access to both preceding and following context simultaneously. At each timestep t, the BiLSTM produces:

$$\mathbf{h}_t = \left[\overrightarrow{\mathbf{h}}_t;\ \overleftarrow{\mathbf{h}}_t\right] \in \mathbb{R}^{512}$$

TABLE VII
REGULARISATION TECHNIQUES AND BIAS–VARIANCE EFFECTS.

Technique	Effect
Dropout	Reduces variance by preventing neuron co-adaptation
AdamW decay	Shrinks weights ⇒ smoother boundary ⇒ reduced variance
Early stopping	Stops training before variance dominates
LayerNorm	Reduces activation variance, stabilises gradients
Layer freezing	Reduces effective complexity ⇒ lower variance

B. Self-Attention vs. Last Hidden State

Using the final LSTM hidden state assumes the last words are most predictive—which is not generally true. Self-attention learns a distribution $\boldsymbol{\alpha} \in \mathbb{R}^T$ over all timesteps. The context vector $\mathbf{c} = \sum_t \alpha_t \mathbf{h}_t$ concentrates on high-signal positions.

C. Transfer Learning with DistilBERT

Training a large Transformer from scratch on 72,000 samples would result in severe overfitting. DistilBERT's pre-trained weights encode general English language knowledge from 3.3 billion words. Fine-tuning only requires adapting already-rich representations to the sales domain.

D. Cross-Entropy Loss

For K-class classification (K=2):

$$\mathcal{L} = -\sum_k y_k \log p_k$$

where y_k is the one-hot true label and $p_k = \mathrm{softmax}(\ell_k)$. For the binary case:

$$\mathcal{L} = -\big[y \log p + (1-y)\log(1-p)\big]$$

This loss penalises confident wrong predictions more than uncertain wrong predictions, driving the model toward well-calibrated probability estimates.

E. Bias–Variance in Deep Learning

Table VII summarises how regularisation techniques affect the bias-variance trade-off.

Architecture choices also affect bias: the 2-layer BiLSTM provides lower bias (more capacity for complex sequential patterns) than a single-layer variant, and DistilBERT with 6 Transformer layers has very low bias due to its rich pre-trained language structure.

VIII. SOFTWARE ARCHITECTURE

The `src/deep_learning.py` module contains all deep learning components: `TokenizerConfig`, `LSTMConfig`, `TransformerConfig`, and `TrainingConfig` dataclasses (no magic numbers); `SalesTokenizer` for word-level tokenisation; `TextClassificationDataset` and `TransformerDataset` PyTorch Dataset classes; `SelfAttention` (Bahdanau-style module); `BiLSTMClassifier` and `DistilBERTClassifier` model classes with explicit weight initialisation; `EarlyStopping` callback; `get_device()` for auto-detection; and dedicated `train_*` / `evaluate_*` functions for each architecture.

Reproducibility is ensured through: saved preprocessed tensors, best-model checkpoints (`model_lstm_best.pt`, `model_distilbert_best.pt`), consistent `random_state=42`, and all configuration defined in dataclasses.

IX. CONCLUSION

Phase 2 successfully extends the sales prediction pipeline with two deep learning architectures operating on raw text. The BiLSTM with self-attention demonstrates principled sequential modelling with orthogonal LSTM initialisation, forget-gate bias = 1, and learned attention pooling. DistilBERT provides efficient Transformer-based transfer learning with differential learning rates and selective layer freezing. The full regularisation suite (dropout, LayerNorm, early stopping, gradient clipping, AdamW weight decay, ReduceLROnPlateau) ensures both models generalise rather than memorise.

Phase 2 models require significantly more computation than Phase 1 (hours vs. seconds), but operate on raw text without any manual feature engineering—making them more adaptable to new domains. The cross-phase comparison on the same 20,000-sample test set provides a fair evaluation of feature engineering + classical ML (Phase 1) against end-to-end text modelling (Phase 2), with direct implications for choosing the right approach for a given production scenario.

Future work includes pretrained embedding initialisation for BiLSTM, account-level train/test splits, probability calibration, and ensemble methods combining Phase 1 features with Phase 2 representations.

ACKNOWLEDGMENT

We thank the HuggingFace dataset contributors [1] and the open-source communities behind PyTorch [**?**] and scikit-learn [**?**].

REFERENCES

[1] Hugging Face, "Dataset Card: DeepMostInnovations/saas-sales-conversations," accessed via project pipeline. [Online]. Available: https://huggingface.co/datasets/DeepMostInnovations/saas-sales-conversations

[2] R. Soni, A. Verma, and S. Kumar, "Intelligent Sales Prediction Using Machine Learning Techniques," *Int. J. Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 2, pp. 112–121, 2024.

[3] M. Das, S. Kamalanathan, and P. J. A. Alphonse, "A Comparative Study on TF-IDF Feature Weighting Method and its Analysis using Unstructured Dataset," arXiv preprint arXiv:2308.04037, Aug. 2023. [Online]. Available: https://arxiv.org/abs/2308.04037

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762